AI Copilot for Manufacturing Process Planning & Quality Inspection

Concepts, Architecture & Flowcharts – A Learning Guide

Prepared as a self-study companion to the codebase

September 2026

Abstract

This guide explains *every concept* used to build the AI Manufacturing Copilot – retrieval-augmented generation (RAG), embeddings, vector search, structured LLM output, deterministic validation, vision inspection, human-in-the-loop workflows, JWT authentication, and more – each in layman's terms first, then mapped to the actual files in this repository, with flowcharts. Read it end-to-end and you will understand not just *what* the system does, but *why* each piece exists and *where* it lives in the code.

Contents

1 The Big Picture: What Are We Building and Why? 3
- 1.1 The problem, in a story . . . 3
- 1.2 The solution . . . 3
- 1.3 System flowchart (the 10,000-meter view) . . . 3
- 1.4 Where everything lives . . . 4

2 Retrieval-Augmented Generation (RAG) 4
- 2.1 The concept . . . 4
- 2.2 Why this kills hallucination (mostly) . . . 4
- 2.3 The RAG pipeline in this codebase . . . 4
- 2.4 Chunking: why size matters . . . 5

3 Embeddings & Vector Search 5
- 3.1 The concept . . . 5
- 3.2 Two providers, one interface . . . 5
- 3.3 Calibration: why we rescale hashing scores . . . 5
- 3.4 Vector stores . . . 5
- 3.5 Hybrid retrieval . . . 6

4 LLMs, Structured Output & the Planner 6
- 4.1 The concept of an LLM gateway . . . 6
- 4.2 Structured output: the single most important LLM trick . . . 6
- 4.3 The planner pipeline . . . 6
- 4.4 The refusal gate: when the assistant says "I don't know" . . . 7
- 4.5 The offline FakeLLM . . . 7

5 Deterministic Validation: The Rule Engine 7
- 5.1 The concept . . . 7
- 5.2 What the rules check . . . 7
- 5.3 The repair loop . . . 7

6 Vision Inspection: Quality Gate + VLM **8**
6.1 The concept . . . 8
6.2 The quality gate (pure classical CV, no AI) . . . 8
6.3 The analysis pipeline . . . 8
6.4 Calibrated thresholds and evidence . . . 8

7 Human-in-the-Loop & Traceability **9**
7.1 The state machine . . . 9
7.2 The audit trail . . . 9
7.3 Feedback: learning from corrections . . . 9

8 Security & Users **9**
8.1 JWT authentication in plain words . . . 9
8.2 Roles . . . 9
8.3 Defense in depth (the challenges table, implemented) . . . 10

9 Production Engineering Practices Used **10**

10 Glossary (Jargon → Plain Words) **10**

1 The Big Picture: What Are We Building and Why?

1.1 The problem, in a story

Imagine a factory that makes precision shafts. When a customer orders 25 shafts of a new type, a senior process engineer must write a **process plan**: an ordered list of machining operations (turn, mill, drill, grind…), the machine and tools for each, cutting speeds, and the quality checks. This knowledge lives in the heads of a few veterans and in scattered PDFs – SOPs, routing sheets, quality standards, past lessons-learned. Meanwhile, quality inspectors photograph parts and judge defects by eye, and their findings sit in a separate system nobody cross-references.

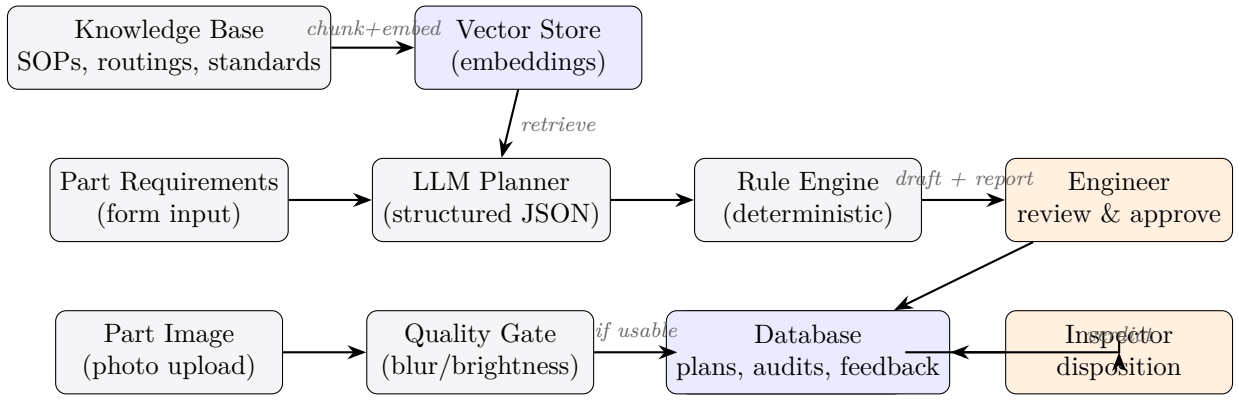
In plain words: The company's expertise is locked in silos: people, PDFs, and photos. New or busy engineers can't easily reuse it, and quality data never feeds back into planning. The result: slow planning, repeated mistakes, defects escaping to customers.

1.2 The solution

Build a **copilot** (an assistant, not an autopilot) that:

1. **Reads the plant's knowledge** (SOPs, routings, standards, lessons) and can answer questions about it *with citations* – this is RAG.
2. **Drafts process plans** from part requirements, grounded in that knowledge, and *validates them with deterministic rules.*
3. **Inspects part images** with a vision model, but only after the image passes a strict quality gate.
4. **Never ships anything on its own** – an engineer must approve every plan and disposition every part, and every action is audit-logged.

1.3 System flowchart (the 10,000-meter view)



1.4 Where everything lives

Concern	Code
Settings (all knobs)	`backend/app/core/config.py`
Database tables	`backend/app/models.py`
API request/response shapes	`backend/app/schemas.py`
Chunking	`backend/app/services/chunker.py`
Embeddings (real + offline)	`backend/app/services/embeddings.py`
Vector store	`backend/app/services/vector_store.py`
Retrieval + citations	`backend/app/services/knowledge.py`
LLM gateway	`backend/app/services/llm.py`
Plan generation	`backend/app/services/planner.py`
Rule validation	`backend/app/services/rules.py`
Image quality gate + VLM	`backend/app/services/inspection.py`
Q&A with refusal	`backend/app/services/qa.py`
Auth + JWT	`backend/app/core/security.py`, `app/api/deps.py`
API routes	`backend/app/api/routes/*.py`
Frontend pages	`frontend/src/pages/*.tsx`
Rule packs (editable by engineers)	`data/rulepacks/*.yaml`

2 Retrieval-Augmented Generation (RAG)

2.1 The concept

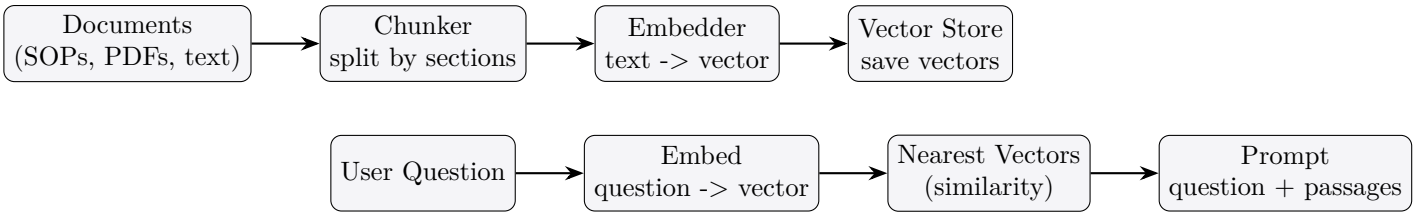
An LLM is a brilliant intern with *amnesia*: it read the whole internet during training, but it has never seen *your* SOPs, and it will confidently make things up when unsure. **RAG** fixes this: *before* asking the model to answer, you *retrieve* relevant passages from your own documents and paste them into the prompt with one instruction: "answer using ONLY these passages, and cite them."

In plain words: Instead of asking a know-it-all stranger, you hand the expert the relevant manual pages *open at the right page*, and ask them to answer while pointing at the page for every claim. If the answer isn't on those pages, they must say so.

2.2 Why this kills hallucination (mostly)

- The model isn't recalling – it's *reading*. Invention becomes unnecessary.
- Every statement can be tied to a chunk ID the user can click and verify.
- If retrieval finds nothing relevant, the system *refuses* rather than guesses (see §4.4).

2.3 The RAG pipeline in this codebase



Indexing path (runs once per document): `knowledge.ingest_document()` in `backend/app/services/knowledge` calls `chunker.chunk_text()`, `embeddings.embed()`, and `vector_store.upsert()`.

Query path (runs per question): `knowledge.retrieve()` embeds the question, asks the store for nearest chunks, **blends** the similarity score with a keyword-overlap score (0.8/0.2), and returns `Citation` objects (document, version, section, score, snippet).

2.4 Chunking: why size matters

Embeddings turn a chunk into one vector – so a chunk that mixes five topics becomes a mushy average. Our chunker (`services/chunker.py`) splits on *headings* first ("SECTION 3: CUTTING PARAMETERS"), packs paragraphs up to ≈1200 characters with 150 characters of overlap between consecutive chunks, and records the section name as metadata – so citations can say "SOP-MILL-001, Section 3."

In plain words: Like filing: don't stuff the whole manual into one folder. Cut it at chapter boundaries so each folder is about one thing, and keep the chapter title on the tab.

3 Embeddings & Vector Search

3.1 The concept

An **embedding** is a list of numbers (a vector) that represents a text's *meaning*. Texts with similar meanings get vectors that point in similar directions. The similarity of two texts is the **cosine** of the angle between their vectors: 1.0 = identical meaning, 0.0 = unrelated.

In plain words: Think of a map where every sentence is a city. "How fast can I mill steel?" and "Roughing steel: 3000–5000 RPM" are two streets in the same neighborhood; "Employee parking policy" is in another state. We find answers by searching the neighborhood, not by reading every page.

3.2 Two providers, one interface

`services/embeddings.py` defines an abstract `BaseEmbeddings` with two implementations:

- **OpenAICompatibleEmbeddings** – calls any `/v1/embeddings` endpoint (OpenAI, Azure, vLLM, Ollama). Production quality.
- **HashingEmbeddings** – a zero-dependency offline fallback: it hashes each word/bigram into buckets to build a vector. It knows *words*, not *meaning*. Used in dev/tests so the whole system runs with no API key.

In plain words: The real model understands that "RPM" and "spindle speed" are related. The hashing fallback is a hyper-literal librarian: it only matches when you use the same words. Both plug into the same socket, so switching is one line in `.env`.

3.3 Calibration: why we rescale hashing scores

Real semantic models give meaningful matches cosine scores like 0.4–0.8, and garbage around 0.1–0.2. Hashing vectors give meaningful matches around 0.25–0.33 and garbage 0.10–0.15 – a *compressed band*. A fixed threshold like 0.25 would behave completely differently on the two providers. So in `knowledge.retrieve()` we rescale the hashing band onto [0,1] (a linear map, like converting Celsius to Fahrenheit), so one threshold works everywhere. The lesson generalizes: **when swapping models, recalibrate thresholds – score scales are model-specific, not universal**.

3.4 Vector stores

`services/vector_store.py` offers:

- **NumpyVectorStore** – exact cosine search, in-memory, rebuilt from the database at startup. Perfect for one server and a few thousand chunks.
- **ChromaVectorStore** – optional persistent ANN store (`pip install .[chroma]`) for larger corpora.

Both implement `upsert / delete_document / search / count` – the rest of the system never knows which is running. *Program to an interface* is the pattern.

3.5 Hybrid retrieval

Dense similarity misses exact keywords ("h6 tolerance"), and keyword matching misses paraphrases. We do both and blend: final $= 0.8 \times$ dense $+ 0.2 \times$ keyword overlap. This is called **hybrid search** and is what real production RAG systems do.

4 LLMs, Structured Output & the Planner

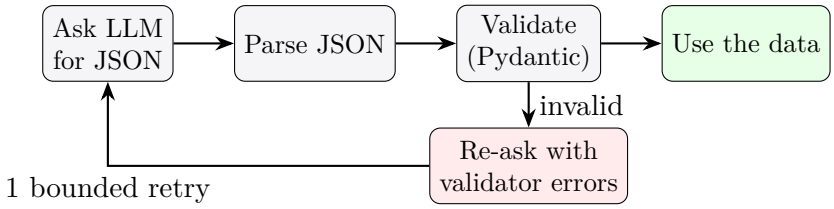
4.1 The concept of an LLM gateway

Different providers (OpenAI, Azure, vLLM, Ollama) all speak a nearly identical "chat completions" dialect. `services/llm.py` wraps that dialect behind one interface, so the planner/Q&A/inspection code never says "OpenAI" – it says "`llm.complete_json(...)`". Swapping providers is a `.env` edit, not a code edit.

4.2 Structured output: the single most important LLM trick

A plan must be *data* (operations, machines, parameters), not an essay – data can be validated, stored, versioned, and shown in a UI. The trick:

1. Define a Pydantic schema (`services/plan_schema.py`): `GeneratedPlan` with `operations[]`, each having `op_number, machine, tooling, parameters…`.
2. Instruct the model: "Output ONLY JSON matching this shape."
3. Parse the reply, extract the JSON (even from markdown fences), *validate against the schema.*
4. If validation fails, send the errors back: "Your JSON failed validation because X. Return corrected JSON only." (One bounded retry.)



In plain words: It's a tax form. You don't accept a free-text letter; you reject the form, circle the wrong fields, and hand it back once. After one retry you escalate to a human. This converts "the model said something" into "the data passed the schema."

This loop lives in `llm.complete_json()`.

4.3 The planner pipeline

`services/planner.py` orchestrates five steps:

1. **Retrieve** knowledge for the part (SOPs, routings, standards, lessons) → numbered context blocks `[K1]..[Kn]`.
2. **Prompt** the LLM: requirements + context + "cite the [Kn] you use."
3. **Structured generation** via `complete_json`.
4. **Rule validation** (§5); if errors, one repair round with the rule violations fed back to the model.
5. **Grounding report**: count operations carrying citations; scan for numeric specs ("0.1 mm", "1800 RPM") that appear in *no* retrieved chunk → flagged as **unverified claims** shown to the engineer.

4.4 The refusal gate: when the assistant says "I don't know"

The Q&A endpoint refuses to answer when retrieval shows nothing relevant. Two checks (`services/qa.py`):

1. **Absolute floor**: the best chunk's calibrated score is below a threshold – nothing even came close.
2. **Separation**: compare the best chunk's score to the *average* score across the whole corpus for this query. A question the corpus can answer produces one neighborhood clearly above the background (separation $\gg 1$); a question about nothing in the corpus scores uniformly – nothing stands out (separation ≈ 1) $\Rightarrow$ refuse.

In plain words: Before answering, the assistant glances at the whole library: if one book lights up for this question, answer from it; if everything is equally dim, say "this isn't in the library" instead of improvising. That self-comparison makes the threshold work for *any* embedding model without manual tuning.

4.5 The offline FakeLLM

`FakeLLM` returns deterministic canned JSON (plan, inspection, or Q&A, routed by prompt markers). This is not a toy: it makes the *entire product* – auth, RAG, planning, validation, approval, UI – testable and demonstrable with zero API cost, and makes the 29 automated tests hermetic (no network flakiness).

5 Deterministic Validation: The Rule Engine

5.1 The concept

LLMs propose; rules dispose. After generation, plain Python code re-checks the plan against plant master data. No probability involved – the same input always yields the same verdict. This is how you get *auditability*: "why was this plan rejected?" has a precise answer: "rule MACH_FORBIDDEN: milling is not allowed on CNC Lathe-1."

In plain words: The AI is a talented intern writing the plan; the rule engine is the grumpy but reliable inspector with the checklist who stamps it. Interns get better with feedback; the checklist never gets tired or creative.

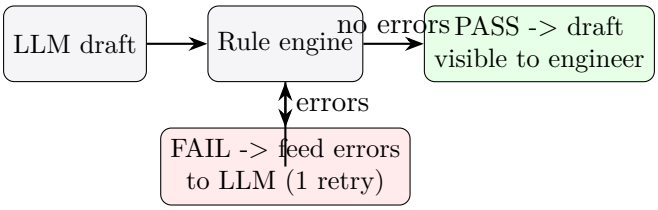
5.2 What the rules check

Rules live in **YAML** (`data/rulepacks/shop_rules.yaml`) so process engineers can edit them *without touching code*:

- **Structural**: op numbers strictly increasing, every op has a machine, an inspection plan exists, checkpoints present (warnings).
- **Machine capability**: machine must exist in approved master data (`MACH_UNKNOWN`); process must be permitted there (`MACH_FORBIDDEN`); parameters within the approved list.
- **Material**: forbidden processes per material; required notes ("material certification") present – else warning.
- **Quantity policy**: orders $\leq$10 pcs require 100% visual inspection in the plan (`QTY_INSPECT`).

5.3 The repair loop

If errors exist, the planner sends them back to the LLM once: "fix these rule violations and return the full corrected JSON." If it still fails, the plan is saved *with its errors visible* – never silently fixed, never silently accepted. The engineer sees `RuleValidationReport` with rule IDs, severity, and messages.



6 Vision Inspection: Quality Gate + VLM

6.1 The concept

A **VLM** (vision-language model) reads an image plus a text prompt and answers in text: "there is a scratch in the upper-left quadrant, confidence 0.8." But models are only as good as their input – a blurry, dark, or glared photo produces confident nonsense. So before the VLM ever runs, deterministic **image quality metrics** decide whether the image is analyzable at all.

6.2 The quality gate (pure classical CV, no AI)

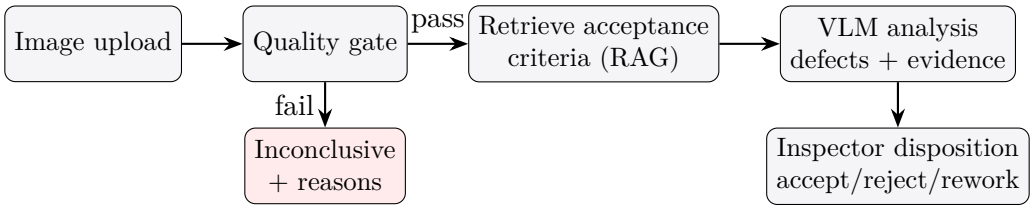
`services/inspection.py::assess_quality()` computes four metrics with OpenCV:

- **Resolution**: width/height minimums (can we see detail at all?).
- **Brightness**: mean pixel intensity within bounds (not pitch-black, not washed out).
- **Sharpness**: variance of the Laplacian – blur smears edges, which lowers this variance dramatically (the classic blur detector).
- **Glare**: fraction of near-saturated pixels (specular reflections hide the surface underneath).

Failed gate ⇒ verdict **inconclusive** with the exact failed checks listed. *"We cannot judge this image" is the honest answer* – challenge #2 from the project brief, solved with arithmetic instead of hope.

In plain words: The bouncer checks ID before letting anyone into the club. Dark, blurry, or glared photos are turned away at the door with a reason – they never reach the (overconfident) AI inside.

6.3 The analysis pipeline



Note the second RAG use: before analysis we retrieve the *visual acceptance criteria* (e.g., QS-77: "scratches >5mm on sealing faces = reject") and paste them into the VLM prompt – so defect judgments reference plant policy, not model vibes.

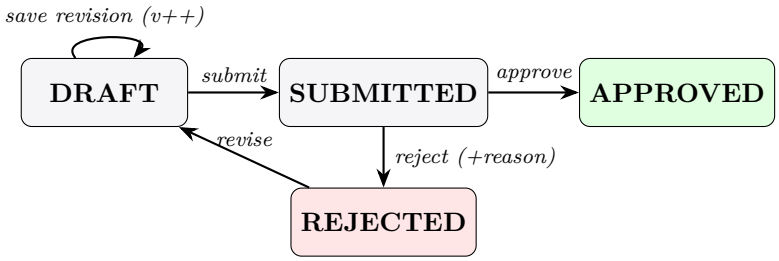
6.4 Calibrated thresholds and evidence

Every defect carries a **confidence**. Below-threshold findings are kept but marked `below_threshold: true` – shown to the human differently rather than hidden. Each defect needs **evidence** ("linear mark consistent with handling, does not catch fingernail") – because an inspector must be able to *disagree* with the AI, and the audit trail records who decided what.

7 Human-in-the-Loop & Traceability

7.1 The state machine

Nothing generated is ever "final". A plan is a **state machine** (`models.PlanStatus`, enforced in `routes/plans.py`):



Illegal transitions are rejected by the API (you cannot approve a draft – first it must be submitted). Every transition writes an `Approval` row: who, what, when, comment, and a **snapshot of the plan as it was reviewed**. The DB rejects the write if a transition is invalid – the database itself is the last line of defense.

7.2 The audit trail

Every meaningful action (plan generated, decision taken, disposition set, document ingested) appends to `audit_logs` with actor, action, target, and JSON detail. Admins can read the tail via `/admin/audit`. This is the "black box" that answers "who did what, when, and based on which sources" – the traceability objective from the brief.

7.3 Feedback: learning from corrections

Every plan and inspection can be rated 1–5 with notes and structured corrections (`feedback` table). Approving with edits, rejecting with reasons, accepting or reworking parts – all become *training signal* for prompt/RAG improvements. This is the "Manufacturing Memory" innovation: the system compounds the value of every human decision instead of discarding it.

8 Security & Users

8.1 JWT authentication in plain words

Login once ⇒ server issues two signed tokens: an **access token** (short-lived, 30 min, sent as `Authorization: Bearer ...` on every API call) and a **refresh token** (7 days, used once to get a new pair, then invalidated – "rotation"). The server signs tokens with a secret key; it never stores passwords in readable form (bcrypt hash). Anyone can *read* a token; nobody can *forge* one without the secret.

In plain words: Access token = a day pass stamped by the guard; refresh token = your membership card that lets you get a new day pass at the desk – once per card, so a stolen card can't be reused silently.

8.2 Roles

Three roles: **admin** (user management, audit), **engineer** (generate, approve, inspect, ingest), **viewer** (read-only). Enforced by a dependency (`require_roles`) on each route; the first registered user becomes admin.

8.3 Defense in depth (the challenges table, implemented)

Threat	Mitigation in this codebase
Hallucinated advice	RAG + citations + schema-validated output + unsupported-claim scan + human approval
Bad images	Deterministic quality gate with explicit failure reasons
Rule violations	YAML rule packs + repair loop + never-silently-accept
Unauthorized access	JWT + roles + bcrypt + refresh rotation
Abuse	Per-IP rate limiting middleware
Lost accountability	Audit log + approval snapshots + versioned documents
Secret leakage	`.env` outside git; `.env.example` documents every key

9 Production Engineering Practices Used

12-factor config. Every knob is an environment variable with a safe default (`core/config.py`); no secrets in code; `.env.example` is the contract.

Error taxonomy. One exception class per failure type (`core/errors.py`) mapped to HTTP codes; clients get `{error: {code, message}}` – never a stack trace.

Structured logs + request IDs. JSON logs with a request-id contextvar (`core/logging.py`, `core/middleware.py`); grep-able and traceable across services.

Health & metrics. `/health` reports provider and DB status; `/metrics` exposes Prometheus counters – what a load balancer and a dashboard need.

Interface swappability. LLM, embeddings, vector store are each behind a tiny abstract base – providers are .env choices, not rewrites.

Hermetic tests. 29 tests cover auth, ingestion, retrieval, refusal, planning, rules, inspection gate, dispositions, feedback, dashboard, audit – all offline, deterministic, fast (seconds).

Containers. `Dockerfile.backend/.frontend` + `docker-compose.yml / .prod.yml` give one-command deploys with Postgres, healthchecks, and an nginx reverse proxy with security headers.

10 Glossary (Jargon → Plain Words)

Term	What it actually means
LLM	Large Language Model – a neural network trained on text that predicts plausible continuations. Brilliant, occasionally wrong with total confidence.
VLM	The same idea for images: picture + question in, description out.
RAG	Retrieve relevant passages first, then have the LLM answer *only from them*, citing them.
Embedding	A list of numbers representing a text's meaning; similar meaning = similar numbers (nearby vector).
Cosine similarity	1− angle between two vectors; 1 = same meaning, 0 = unrelated.
Vector store	A database that finds nearest vectors fast.
Chunk	A paragraph-sized piece of a document; the unit of retrieval and citation.
Token	A word-piece; how LLMs and embeddings split text (and how they charge you).
Prompt	The full text sent to the model: instructions + context + question.
System prompt	The standing instructions defining the model's role and rules.

Hallucination	The model inventing facts. Cured here by grounding, citations, refusal.
JSON schema	The exact required structure of an answer; used to force usable data out of the model.
Rule engine	Deterministic Python checks against plant master data; no AI involved.
Quality gate	Pre-AI arithmetic checks (blur/brightness/glare) that refuse unusable images.
Disposition	The human decision on a part: accept / reject / rework.
JWT	A signed token proving who you are; contains claims (user id, role, expiry).
bcrypt	A slow, salted password hash; passwords are never stored in plain text.
CORS	Browser rule for which websites may call the API; configured via env.
Rate limiting	Cap requests per minute per client to survive abuse or bugs.
ANN	Approximate Nearest Neighbor – vector search that trades a little accuracy for a lot of speed.
Migration	A versioned change to the DB schema (Alembic); tables evolve without data loss.
Idempotent	Running it twice has the same effect as once (seed script, ingestion).

How to study this codebase in one evening

1. Read `backend/app/core/config.py` – see every knob with its default.
2. Read `services/knowledge.py` – indexing and retrieval, 200 lines.
3. Read `services/planner.py` – the 5-step pipeline and its prompts.
4. Read `services/rules.py` + `data/rulepacks/shop_rules.yaml` – how plants encode expertise.
5. Read `services/inspection.py::assess_quality` – 40 lines of honest computer vision.
6. Run `pytest` and read the tests – they are executable specifications.
7. Click through the UI with the demo account and watch the API logs stream.

The core lesson of this architecture: LLMs are used where language matters, arithmetic where truth matters, and humans where judgment matters – and every layer can show its work.